Development of a MATLAB-Based Simulation System for a 3-Degree-of-Freedom Robotic Arm for Studying Forward Kinematics and Inverse Kinematics

K. THANAWUT, W. PONTAKORN, S. WEERASIT, L. APHISIT
School of Engineering, Department of Instrumentation and Control Engineering
King Mongkut's Institute of Technology Ladkrabang
1 Chalong Krung 1 Alley, Lat Krabang, Bangkok, Thailand

Abstract—This paper presents a MATLAB-based simulation system for studying Forward Kinematics and Inverse Kinematics of a three-degree-of-freedom robotic arm. A serial model with three revolute joints is constructed using Robotics System Toolbox, while a custom geometric solver calculates joint angles from Cartesian targets. The interface combines numerical inputs, sliders, selectable elbow branches, coordinate frames, and a homogeneous transformation matrix display. Verification follows the worked examples in the project report: Forward Kinematics is compared with hand calculations, and the angles returned by Inverse Kinematics are checked by substitution into Forward Kinematics. For joint angles of 30, −30, and 60 degrees, the end-effector position is approximately 0.300000, 0.173205, and 0.270000 meters. Two admissible elbow configurations reach this position with different end-effector orientations. The transformation matrix connects these numerical results with the displayed coordinate frames. A laboratory worksheet organizes learning into robot structure, frame assignment, kinematic calculation, and MATLAB verification. The results demonstrate consistency for the reported examples; physical robot accuracy and learning effectiveness have not been evaluated.

Keywords—robot simulation, MATLAB, Forward Kinematics, Inverse Kinematics, homogeneous transformation, robotics education

I. INTRODUCTION

Understanding a robotic arm requires connecting joint motion with the position and orientation of its end effector. This connection is commonly expressed through coordinate frames and homogeneous transformations [1]. For a beginner, however, a correct matrix calculation may still be difficult to relate to the visible arrangement of the links. An interactive model can make the effect of an input change observable alongside the corresponding numerical result.

Computational treatments of robotics combine mathematical descriptions with executable examples [2]. MATLAB provides a rigid-body representation in which joints define motion relative to parent bodies [3], and transformations can be evaluated for a specified configuration [4]. These facilities provide a basis for an instructional application in which students can compare their own calculations with a consistent robot model.

This work develops such an application for a three-revolute-joint arm. Its contribution is the integration of a geometric Inverse Kinematics solver, synchronized joint and Cartesian controls, frame visualization, and a matrix display. The study presents the development process and the hand-calculation comparisons documented in the project report. It does not propose a new kinematics algorithm or report a controlled study of student learning.

The scope is deliberately limited to positional kinematics. A three-joint mechanism is sufficient to illustrate base rotation, coupled shoulder and elbow motion, and multiple positional solutions without introducing a wrist. Orientation is still calculated and displayed, but it is a consequence of the selected joint configuration rather than an independently specified target. This distinction is important when students interpret the six-dimensional concept of a pose.

The application is organized around three practical questions. First, does a joint configuration produce the same position in the geometric equations and the rigid-body model? Second, when a Cartesian target is specified, does the selected branch yield an admissible joint vector whose computed position matches that target? Third, do the graphical frames and transformation table use the same coordinate convention? These questions determine the implementation and verification procedures used in this paper.

The paper first defines the mechanism and its coordinate convention, then describes the mathematical formulation and software implementation. Verification follows the report's worked examples for Forward Kinematics, the two Inverse Kinematics branches, and the transformation matrix. The final discussion describes the laboratory worksheet and the development directions identified in the report.

II. ROBOT MODEL AND COORDINATE CONVENTION

The arm consists of a fixed base, a vertical first link, an upper arm, and a forearm. The joint variables q_1, q_2, and q_3 represent base yaw, shoulder rotation, and elbow rotation relative to the upper arm. Table I lists the dimensions and imposed joint limits. The shoulder height is H = baseHeight + L_1 = 0.25 m. Fig. 1 shows the three-dimensional arm at the worked-example configuration.

TABLE I. ROBOT PARAMETERS

Parameter	Value
Base height	0.07 m
L_1	0.18 m
L_2	0.22 m
L_3	0.18 m
q_1	−180° to +180°
q_2	−90° to +90°
q_3	−120° to +120°

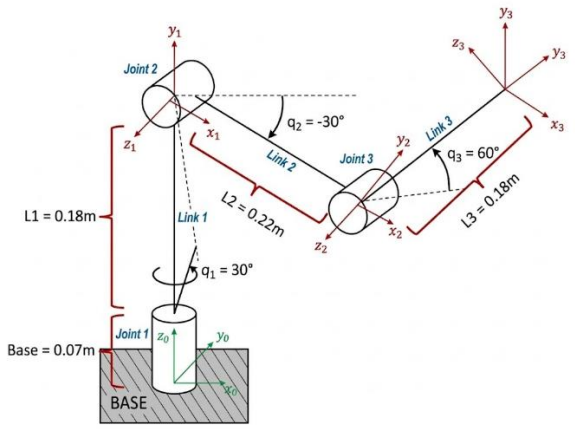


Fig. 1. Model geometry at $q_1 = 30°$, $q_2 = -30°$, and $q_3 = 60°$. Arcs start from the horizontal radial direction for q_2 and the upper-arm extension for q_3. Joint axes and tool axes follow the model convention.

The world Z axis points upward. The base joint rotates about its local +Z axis; the shoulder and elbow rotate about their local +Y axes. With all angles zero, the arm extends along +X. Positive shoulder and elbow rotations bend a link initially pointing along +X toward −Z. This sign convention is retained in every equation and display.

The rigidBodyTree model uses column configurations and contains base_link, link1, link2, link3, and tool. Only three joints are movable. The base and tool joints are fixed. Joint mounting offsets are, respectively, baseHeight along Z, L_1 along Z, L_2 along X, and L_3 along X. These local body frames are defined directly; they should not be assumed to be Denavit–Hartenberg frames.

A. Frame Placement and Zero Configuration

A frame identifies both an origin and three orthogonal axes. In the zero configuration, base_link is coincident with the root frame, link1 starts at the top of the base, link2 starts at the shoulder, link3 starts at the elbow, and tool is at the end of the forearm. A body frame need not coincide with the geometric center of its rendered solid. The cylinder and box meshes are offset from their frame origins to extend in the intended positive local direction.

For example, the visual base cylinder is centered at half its height so that its bottom lies at world Z = 0. The upper-arm box is centered at local $X = L_2/2$, whereas the elbow joint is mounted at local $X = L_2$. These offsets describe different objects: the first locates the visible solid and the second locates the next joint. Confusing them would shift the graphic or the kinematic chain by half a link length.

TABLE II. BODY CONNECTIONS AND MOUNTING OFFSETS

Body	Parent	Offset and motion
base_link	Root	Identity; fixed
link1	base_link	Z = baseHeight; rotate +Z
link2	link1	Z = L_1; rotate +Y
link3	link2	X = L_2; rotate +Y
tool	link3	X = L_3; fixed

Table II distinguishes translation along a parent-frame axis from rotation about a local joint axis. When the base turns, the shoulder and elbow axes turn with it. They are therefore not fixed world Y axes for every configuration. This is why the complete chain must be evaluated instead of treating all joint rotations as independent rotations about global axes.

B. Degrees of Freedom and Model Assumptions

The five bodies do not imply five degrees of freedom. The two fixed joints add mounting relationships, while only the three revolute joints introduce independent variables. The rigid-body model assumes constant dimensions and ideal joint rotations. It does not model compliance, backlash, motor dynamics, friction, load-dependent deflection, or measurement noise. All distances refer to frame origins rather than the outer surfaces of the rendered solids.

Denavit–Hartenberg parameters provide an alternative systematic description of a serial chain [1]. They are included in the learning activities, but the implementation here sets mounting transforms and joint axes directly. A comparison with a Denavit–Hartenberg derivation is meaningful only after aligning the base frame, terminal frame, angle offsets, and positive rotation directions. Different frame assignments can describe the same mechanism while producing different intermediate matrices.

III. Kinematic Formulation

A. Forward Kinematics

Using the notation of the project report, r denotes the signed horizontal projection in Forward Kinematics, and H = baseHeight + L_1. Resolving the two arm links gives

$$r = L_2 \cos q_2 + L_3 \cos(q_2 + q_3), \tag{1}$$

$$x = r \cos q_1, \qquad y = r \sin q_1, \tag{2}$$

$$z = H - L_2 \sin q_2 - L_3 \sin(q_2 + q_3). \tag{3}$$

The sum $q_2 + q_3$ occurs because the elbow angle is measured relative to the upper arm. The negative terms in (3) follow from the local +Y rotation convention. Distances are in meters. The interface accepts degrees, whereas trigonometric calculations and robot configurations use radians.

The transform T maps homogeneous coordinates from the tool frame into the base frame. Its upper-left block R describes orientation and its last column contains position $p = [x, y, z]^T$ [1]. For this model, the successive mounting translations and joint rotations give

$$T = Tz(\text{baseHeight})\, Rz(q_1)\, Tz(L_1)\, Ry(q_2) \times Tx(L_2)\, Ry(q_3)\, Tx(L_3). \tag{4}$$

Here Tx and Tz are homogeneous translations, and Ry and Rz are homogeneous rotations about the indicated local axes. The rotational block reduces to $R = Rz(q_1)Ry(q_2 + q_3)$. In the application, getTransform(robot, deg2rad(q(:)), 'tool') evaluates the tool-to-base transform; T(1:3,4) supplies the displayed position [4].

B. Interpreting the Rotation and Position Blocks

The homogeneous matrix can be partitioned into a 3 × 3 rotation block R and a 3 × 1 translation vector p, followed by the bottom row [0, 0, 0, 1]. For a point expressed in tool coordinates, multiplication by T expresses the same point in base coordinates. In particular, the tool origin has local coordinates $[0, 0, 0, 1]^T$, so multiplication selects the final

column. This explains why the first three entries of that column are the end-effector position.

Writing $c_1 = \cos q_1$, $s_1 = \sin q_1$, $c_{23} = \cos(q_2 + q_3)$, and $s_{23} = \sin(q_2 + q_3)$, the rows of the rotation matrix are $[c_1c_{23}, -s_1, c_1s_{23}]$, $[s_1c_{23}, c_1, s_1s_{23}]$, and $[-s_{23}, 0, c_{23}]$. The first column points along the tool X axis in base coordinates, the second along its Y axis, and the third along its Z axis. These are direction vectors, not additional end-effector positions.

The zero-angle case offers a useful diagnostic: R is the identity matrix and $p = [L_2 + L_3, 0, H]^T$. A rotation of the base by 90 degrees transfers the horizontal reach from +X to +Y. A shoulder rotation changes both horizontal reach and height, whereas an elbow rotation changes the direction of the forearm relative to the upper arm. These simple cases help isolate an incorrect sign, axis, or mounting translation.

Forward Kinematics is needed for graphics as well as for reporting the final tool position. The renderer requires the transform of each displayed body, since the shoulder and elbow solids have different origins and orientations. Calculating only the final tool transform would not determine how to place every mesh independently. The implementation therefore evaluates all body frames at the current joint vector and applies each matrix to its own graphics group.

C. Inverse Kinematics and Branch Selection

For a desired position (x, y, z), r now denotes the nonnegative radial distance, as in the report; the signed projection is recovered using radialSign below. The downward displacement is v:

$$r = \sqrt{x^2 + y^2}, \qquad v = H - z. \tag{5}$$

Applying the law of cosines to the upper arm, forearm, and shoulder-to-target distance gives

$$D = \frac{r^2 + v^2 - L_2^2 - L_3^2}{2L_2L_3} \tag{6}$$

Because $D = \cos q_3$, a real geometric solution requires $-1 \le D \le 1$. The implementation rejects $|D| > 1 + 10^{-12}$ and clamps smaller numerical excursions to the valid interval. The code uses branch = 1 for positive q_3 and branch = 2 for negative q_3:

$$q_3 = (3 - 2 \times \text{branch}) \operatorname{acos}(D). \tag{7}$$

For radialSign = +1, $q_1 = \operatorname{atan2}(y, x)$; for radialSign = −1, the solver adds π. If $r < 10^{-12}$ m, it keeps the previous base angle. The report's auxiliary angles α and β give the shoulder angle:

$$\alpha = \operatorname{atan2}(v, \text{radialSign} \times r),$$

$$\beta = \operatorname{atan2}(L_3 \sin q_3, L_2 + L_3 \cos q_3),$$

$$q_2 = \alpha - \beta. \tag{8}$$

Here α is the shoulder-to-target direction measured downward from the signed horizontal axis, and β is the triangle correction caused by the forearm. The report's worked example uses radialSign = +1. In MATLAB, theta1, theta2, and theta3 store q_1, q_2, and q_3 in radians before conversion to degrees.

The radialSign = −1 case represents a base rotation of π combined with a backward arm projection. It is distinct from switching the elbow branch. Candidate angles are converted to degrees, wrapped into a conventional range, and checked against all limits in Table I. A candidate outside any limit is discarded. The function returns an empty result if no candidate survives for the selected elbow sign.

If more than one candidate remains, the solver prefers the candidate nearest to the previous joint vector seed. For candidate j, the wrapped angular difference and selection score are

$$\delta_j = \operatorname{mod}(q_j - \text{seed} + 180°, 360°) - 180°, \tag{9}$$

$$S_j = \delta_{j1}^2 + \delta_{j2}^2 + \delta_{j3}^2. \tag{10}$$

The candidate with minimum S_j is selected. This rule reduces unnecessary changes in configuration, but it does not generate a collision-free path or enforce velocity and acceleration limits.

D. Geometric Feasibility and Numerical Safeguards

The triangle inequality gives an initial geometric distance range of $|L_2 - L_3| \le d \le L_2 + L_3$, where $d = \sqrt{(r^2 + v^2)}$. For the chosen dimensions, this interval is 0.04 to 0.40 m. It assumes unconstrained planar rotation; the actual admissible workspace is smaller because of joint limits. For example, reaching the inner distance of 0.04 m requires a fully folded elbow of 180 degrees, which the imposed ±120-degree elbow limit excludes.

The test on D is performed before evaluating acos. Finite-precision arithmetic can produce a value slightly larger than one for a straight-arm target that is geometrically valid. The tolerance avoids rejecting this case solely because of roundoff. Values outside the interval by more than the tolerance are rejected. Values within the tolerance are clamped, with an additional snap to exactly ±1 near either endpoint. These operations address numerical representation and do not extend the modeled physical reach.

A separate tolerance of 10^{-9} degrees is used when comparing candidates with joint limits. An accepted candidate is then clipped to the exact configured bounds to remove a negligible numerical excursion. Near −180 degrees, the base angle may be represented as +180 degrees when the previous base angle was positive. Both represent the same geometric direction, but this sign choice reduces an unnecessary change in the displayed value at the boundary.

When r is close to zero, Cartesian position does not determine the base yaw uniquely. Preserving the previous yaw supplies a reproducible choice without inventing a preferred direction. At $q_3 = 0$, the positive and negative elbow branches coincide. Thus two menu selections need not produce two distinct configurations at every target. The number of admissible answers depends on geometry, the selected elbow sign, and the configured joint bounds.

E. Candidate Selection and Failure Meaning

The seed is the previous angle vector. It is not a numerical initial guess for an iterative optimizer in this implementation: the candidate angles are calculated analytically. The seed is used to resolve the near-axis yaw and to rank surviving candidates. The squared wrapped-distance score assigns equal

weight to each joint; it does not account for actuator speed, energy, torque, or different mechanical travel costs.

A failure for one elbow sign does not prove that the target is globally unreachable. Another sign may yield a valid configuration. The interface therefore reports that the target is unavailable for the selected elbow within the joint limits. If every candidate is rejected, the solver returns an empty angle vector rather than a partially valid configuration, and the displayed robot retains its previous valid state.

The solver also does not guarantee continuity through singular configurations or around angle-wrap boundaries. Selecting nearby endpoint angles is different from constructing a feasible motion between them. In particular, a short wrapped difference across ±180 degrees may not correspond to an allowed direct motion for a mechanically limited base. The application displays configurations; continuous trajectory planning remains a separate development task.

IV. Software Implementation

Fig. 2 summarizes the computational path for a Cartesian edit. Geometric feasibility is checked before candidates are constructed; joint feasibility is checked afterward. The previous configuration is used only to resolve a nonunique yaw and to rank valid candidates. The selected result then passes through the same Forward Kinematics and rendering path as a direct joint edit.

A. Model Construction and Rendering

Development proceeded through six versions: static robot drawing, numerical joint input, joint sliders, coordinate frames, Cartesian target solving, and the transformation matrix display. The underlying model is built with rigidBodyTree,rigidBody,rigidBodyJoint,setFixedTransform, and addBody. The tool frame is attached at the end of the forearm with a fixed joint.

Rendering is handled by custom cylinder, box, and sphere meshes grouped under hgtransform objects. The modelFrames function obtains body-to-base matrices from getTransform. The drawRobot function assigns these matrices to the corresponding graphics groups. The meshes are created once and reused for subsequent updates, while the camera remains fixed. Thus the graphics and numerical output are driven by the same kinematic configuration.

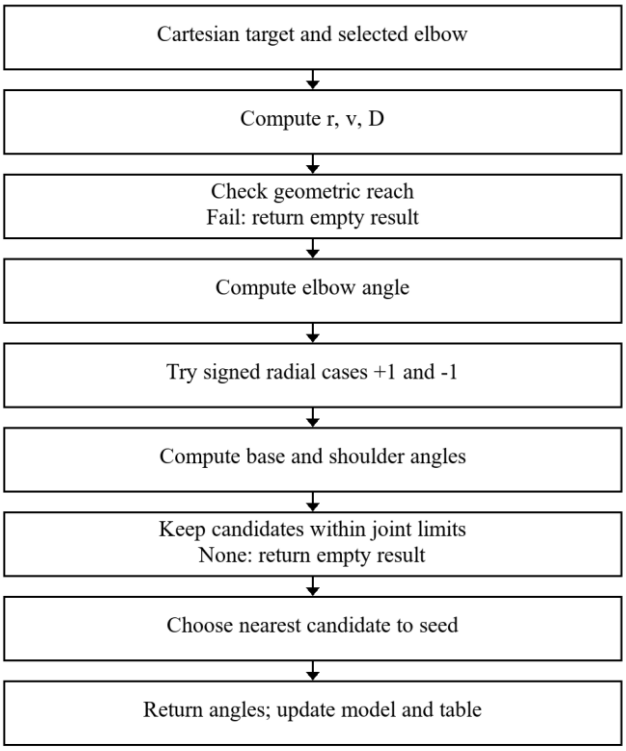


Fig. 2. Processing sequence for a Cartesian target. A failed feasibility check returns an empty result and leaves the displayed joint state unchanged.

B. Interaction and Numerical Feedback

The graphical interface contains three joint input fields and sliders, three Cartesian target controls, an elbow selector, a three-dimensional view, numerical output, and a 4 × 4 transformation table. Joint input triggers Forward Kinematics and synchronizes the Cartesian controls. Cartesian input triggers solveIK; an accepted solution updates the joint controls and then refreshes the model and matrix. Fig. 3 shows the completed interface.

The application uses callbacks rather than a continuous polling loop. During slider dragging, update requests are throttled using a 0.04 s interval; a completed edit requests a final update. This is an implementation setting, not a measured frame-rate guarantee. Nonfinite Cartesian input is rejected by the interface. If a target has no admissible solution, the last valid joint configuration is retained and a status message is displayed.

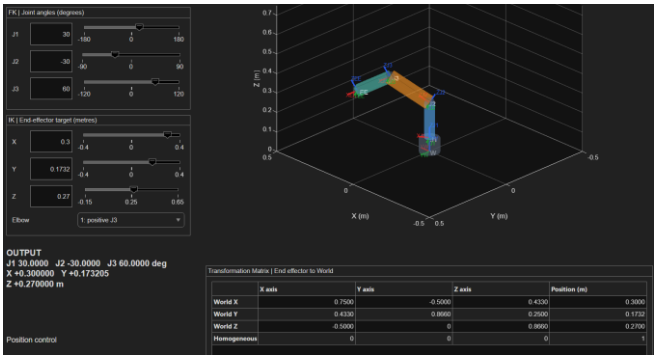


Fig. 3. Positive-elbow configuration in the application. The corresponding position and matrix entries are reproduced in Tables IV–V and (11).

The refresh function reads the current transform, assigns it to transformTable.Data, and formats joint angles and

Cartesian coordinates for display. Coordinate axes attached to the bodies expose the changing frame orientations. Diagnostic output also records the target, D, raw candidate angles, limit decisions, and selected joint changes, allowing the calculation to be traced during development.

C. State Representation and Update Sequence

The robot object stores topology and mounting geometry, while the current joint vector stores the configuration. The scene structure stores graphics handles. Keeping these roles separate avoids interpreting a graphics object as the mathematical model. Within modelFrames, a $4 \times 4 \times 5$ array stores the transformations for base_link, link1, link2, link3, and tool. Its first four slices are used to position the visible rigid-body groups; all five are used for coordinate-frame displays.

After a joint edit, moveJoint changes only the selected angle, synchronizes its input controls, and obtains the resulting tool position. The Cartesian fields are then updated from this position before refresh is called. After a Cartesian edit, moveTarget changes the target coordinate and requests solveTarget. An accepted angle vector replaces the current joint state; an empty result leaves that state unchanged. Both paths end in the same rendering and output routine.

The matrix table is populated from the same T used to extract the displayed position. This avoids maintaining a second independently updated position variable for the output label. The table labels identify the columns as the tool-axis directions and position, with rows expressed in world coordinates. A change in the selected elbow branch can therefore leave the last column unchanged while changing the rotation block, illustrating the difference between position and orientation.

The initial target-slider ranges are based on total arm reach and shoulder height. They are convenient input bounds rather than an exact workspace boundary. The code may expand a slider range to include a typed finite value, but the geometric and joint-limit tests remain responsible for acceptance. Likewise, the plotted Z range begins at zero; this visual clipping is not a collision constraint, and the mathematical model can produce negative heights.

D. Incremental Development and Diagnostics

TABLE III. DEVELOPMENT STAGES

Version	Added capability
1	Robot model and static rendering
2	Numerical joint-angle entry
3	Synchronized joint sliders
4	Body coordinate-frame display
5	Cartesian targets and elbow selection
6	Homogeneous transformation table

Table III summarizes the sequence used to develop the system. Each stage preserves the previous model and adds a focused interface or visualization function. This progression is also suitable for a programming exercise: a learner can first verify the zero configuration, then add input callbacks, then inspect frame motion, and finally compare the two directions of kinematic calculation. Forward Kinematics is already required for rendering before the Cartesian target controls are introduced.

Diagnostic printing is placed inside solveIK to expose r, v, D, branch selection, and each candidate before and after angle wrapping. The output states whether a candidate passed the joint-limit test and reports the selected difference from the previous angles. These messages help trace the calculation during software development; the verification results presented here are the worked examples documented in the report.

E. Essential MATLAB Code

The following excerpts connect the model, equations, and display. They retain the application's variable names, with line breaks adjusted for the column width. Each excerpt belongs to the indicated function and is not a standalone program; initialization, visual geometry, and diagnostic printing are omitted where they do not affect the illustrated operation.

Listing 1. Shoulder-link assembly in makeRobot.

```
body = rigidBody('link2');
joint = rigidBodyJoint('joint2','revolute');
setFixedTransform(joint, ...
    trvec2tform([0 0 param.L1]));
joint.JointAxis = [0 1 0];
joint.PositionLimits = ...
    deg2rad(param.jointLimits(2,:));
body.Joint = joint;
addBody(robot,body,'link1');
```

Listing 1 assumes robot, param, and the parent link1 already exist. Robotics System Toolbox creates the body and revolute joint, places the shoulder L_1 above the link1 origin, sets rotation about local +Y, and attaches link2 to link1. deg2rad converts the configured degree limits to radians. This implements the connection in Table II; addVisual calls define appearance separately.

Listing 2. Forward Kinematics and target extraction in moveJoint.

```
T = getTransform(robot,deg2rad(q(:)),'tool');
target = T(1:3,4).';
```

In Listing 2, q(:) forms a column vector and deg2rad converts the joint angles to radians. The Toolbox function getTransform returns the 4×4 tool-to-base matrix in (4). T(1:3,4) extracts X, Y, and Z; the nonconjugate transpose produces the row vector used by the target controls. This is the program counterpart of (1)–(3).

Listing 3. Geometric calculation excerpts from solveIK.

```
r = hypot(target(1),target(2));
v = param.baseHeight + param.L1 - target(3);
D = (r^2 + v^2 - param.L2^2 - param.L3^2) ...
    / (2*param.L2*param.L3);
if abs(D) > 1 + 1e-12
    return;
end
D = max(-1,min(1,D));
if abs(1-abs(D)) < 1e-12
    D = sign(D);
end
theta3 = (3-2*branch)*acos(D);

% Inside the loop over radialSign = [1 -1]:
theta1 = yaw + (radialSign < 0)*pi;
theta2 = atan2(v,radialSign*r) ...
    - atan2(param.L3*sin(theta3), ...
      param.L2 + param.L3*cos(theta3));
```

Listing 3 implements (5)–(8) using ordinary MATLAB arithmetic and trigonometric functions. The function

initializes q = [] before this excerpt, so return leaves an empty result when the target is beyond geometric reach. branch selects the sign of theta3. yaw is obtained from atan2(y,x), or from the previous base angle near the axis. The final two assignments are evaluated for each radialSign; theta2 directly implements $\alpha - \beta$. The remaining function converts the candidate angles to degrees, checks all joint limits, and selects the valid candidate nearest seed using (9)–(10). Those checks remain necessary before applying a result.

Listing 4. Graphics transforms and matrix-table update.

```
% In drawRobot, after geometry exists:
[frames,bodies] = modelFrames(robot,angles);
for k = 1:4
    scene.bodies(k).Matrix = bodies(:,:,k);
end
for k = 1:5
    scene.frames(k).Matrix = frames(:,:,k);
end

% In refresh:
T = getTransform(robot,deg2rad(q(:)),'tool');
transformTable.Data = T;
```

Listing 4 shows two display stages. The project function modelFrames obtains each body transform through getTransform. Assigning a matrix to a MATLAB hgtransform object moves its existing child geometry or coordinate axes. refresh then writes the tool matrix into the UI table. hgtransform and the table Data property are MATLAB graphics/UI facilities; the kinematic matrix is supplied by Robotics System Toolbox. Thus the displayed robot and matrix are updated from the same joint configuration.

V. Verification and Results

A. Forward Kinematics Comparison

Verification compares independent geometric calculations with the MATLAB model for the same dimensions and angle convention. The principal worked example uses q = [30°, −30°, 60°]. Equations (1)–(3) give r = 0.346410 m and p = $[0.300000, 0.173205, 0.270000]^T$ m. The displayed MATLAB coordinates agree at six decimal places. Table IV summarizes this case and the zero-angle reference configuration visible in the application.

TABLE IV. POSITION COMPARISON

Joint angles (°)	Geometric position (m)	MATLAB position (m)
0, 0, 0	0.400000 0.000000 0.250000	0.400000 0.000000 0.250000
30, −30, 60	0.300000 0.173205 0.270000	0.300000 0.173205 0.270000

Position entries in Table IV are ordered X, Y, Z. Agreement is stated at the reported display precision; it should not be interpreted as measured physical positioning accuracy. The comparison checks whether the mounting offsets, rotation directions, and tool location are consistent with the geometric equations.

B. Inverse Kinematics Round Trip

The target p = $[0.300000, 0.173205, 0.270000]^T$ m is taken from the preceding example. Retaining the unrounded Y value, $\sqrt{3}/10$ m, gives $r^2 = 0.12$ m², v = −0.02 m, and D = 0.5. The elbow choices are therefore +60° and −60°. Substituting each choice into (8) gives the two solutions in Table V. The report screenshots show these configurations and their matching Cartesian outputs.

TABLE V. SOLUTIONS FOR THE SAME TARGET

Elbow sign	q_1 (°)	q_2 (°)	q_3 (°)
Positive	30.0000	−30.0000	60.0000
Negative	30.0000	23.3914	−60.0000

Both solutions satisfy the imposed joint limits and return the same position at six displayed decimal places when evaluated by Forward Kinematics. Their orientations differ because $q_2 + q_3$ differs. The solver constrains only Cartesian position; a three-joint arm cannot in general independently prescribe all three position and all three orientation components.

C. Transformation Matrix Consistency

For the positive-elbow configuration, the analytical transform, rounded to four decimal places, is

$$T = \begin{bmatrix} 0.7500 & -0.5000 & 0.4330 & 0.3000 \\ 0.4330 & 0.8660 & 0.2500 & 0.1732 \\ -0.5000 & 0 & 0.8660 & 0.2700 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (11)$$

These entries agree with the matrix shown in Fig. 3. The last column reproduces the calculated position; the first three columns describe the tool axes in base coordinates. The analytical rotation satisfies $R^TR = I$ and $\det(R) = 1$. Tests of these properties should use unrounded values because display rounding introduces small deviations.

For comparison, the zero-angle transform has an identity rotation and translation $[0.40, 0, 0.25]^T$ m. This reference case confirms that the base height and vertical link are included in the position calculation rather than serving only as visual geometry.

VI. Laboratory Activities and Discussion

A companion worksheet organizes learning into five activities: identifying joints and links and sketching the workspace; assigning frames and Denavit–Hartenberg parameters; calculating Forward Kinematics by matrices and geometry; deriving Inverse Kinematics and comparing elbow branches; and checking numerical results in MATLAB. Students record intermediate calculations and compare them with the displayed configuration and matrix.

Before classroom deployment, the worksheet must use the same toolbox, angle signs, and joint limits as the application. The existing worksheet examples use Peter Corke's Link and SerialLink interface, whereas this application uses MathWorks Robotics System Toolbox. Their modeling commands are not interchangeable. Likewise, a positive-up planar convention must be translated consistently to the positive-down shoulder and elbow convention used here.

The intended teaching value is the opportunity to connect an equation, a numerical answer, and a visible configuration in one activity. The report does not present a pre-test/post-test study or a usability experiment. The simulator and worksheet therefore provide instructional material, while their effectiveness with learners remains a subject for future evaluation.

A. Numerical Display Precision

Reported angles and positions use a limited number of decimal places for readability, whereas the computations retain double precision. Entering a displayed rounded value as a new target can therefore produce a slightly different joint vector. In particular, the displayed Y coordinate 0.1732 m is not exactly $\sqrt{3}/10$ m. Hand and program comparisons should use the same target precision, and comparisons made by reading the interface should be limited to its displayed precision.

For the reported transformation-matrix comparison, the last column is checked against the calculated XYZ position, while the rotation block describes the displayed tool axes. The properties $R^TR = I$ and $\det(R) = 1$ provide mathematical checks on a rotation matrix. These checks complement the coordinate convention and the worked position calculation; they do not establish physical robot accuracy.

B. Worksheet Activities

The first worksheet activity addresses robot structure and workspace. Learners draw the three revolute joints and links, identify the reference frames, and label q_1, q_2, and q_3. The specified dimensions are a 0.07 m base, a 0.18 m vertical link, a 0.22 m upper arm, and a 0.18 m forearm. Top and side views are used to discuss the reachable region. The worksheet specifies q_2 from 0° to 90° and q_3 from −90° to 90°, whereas the application limits are those in Table I; these bounds must be aligned before a direct comparison.

The second activity assigns coordinate frames and Denavit–Hartenberg parameters. Learners identify the joint angle, joint offset, link length, and link twist, then complete the parameter table from the base to the tool. The worksheet asks them to account for the total shoulder height and to check twist-angle signs using the right-hand rule. The resulting table supplies the transformations used in the next activity.

The third activity calculates Forward Kinematics. Learners substitute the parameters into individual transformation matrices A_1, A_2, and A_3 and multiply them in order to obtain the overall transformation. The fourth activity calculates Inverse Kinematics: the base angle follows from the XY projection, the elbow angle from the law of cosines, and the shoulder angle from the side-view triangle. Learners arrange the equations before substituting numerical values.

The fifth activity implements the hand-derived equations and parameters in MATLAB and compares Forward Kinematics and Inverse Kinematics results. The worksheet includes explanatory material, calculation exercises, diagram space, result tables, and incomplete program examples for learners to complete. Its purpose is to connect mathematical reasoning with program execution; no measured improvement in student performance is reported.

C. Limitations and Development Priorities

The report identifies motion between configurations as a development priority. The program could calculate intermediate joint angles, display motion over time, and trace the tool path. This would support comparisons between joint-space motion and paths specified in XYZ coordinates. Continuity of Inverse Kinematics solutions and joint velocity and acceleration limits should be considered to avoid abrupt changes in the simulation.

Collision checking is another proposed extension. Links should be checked against the base, ground, and obstacles, since a mathematically valid position can still cause contact even when every joint angle is within its bounds. Adding these checks would make the simulation constraints more representative of practical operation.

The report also proposes using the revised worksheet and simulator with learners. Evaluation could compare understanding before and after use and collect feedback on explanation clarity, control usability, and activity suitability. Such assessment is future work; the present paper makes no measured learning-outcome claim.

A further direction is connection to a physical arm through motor-control communication and joint-angle measurement, allowing model positions to be compared with measured positions. This requires zero-offset calibration, consideration of component tolerances, and an emergency-stop system. Increasing the number of joints could extend the study to independent position and orientation specification and more complex motion planning and control.

VII. Conclusion

A MATLAB-based simulation system for a three-degree-of-freedom robotic arm was developed with joint and Cartesian input, selectable elbow branches, coordinate-frame visualization, and a transformation matrix display. The reported hand-calculation examples agree with the displayed MATLAB positions. For the representative target, two admissible elbow configurations produce the same position with different tool orientations. The matrix display links the calculated position and orientation to the three-dimensional model. A companion worksheet provides activities on structure, frames, Forward Kinematics, Inverse Kinematics, and MATLAB verification. These results concern the selected mathematical examples and do not establish physical accuracy or learning effectiveness. Future development includes continuous motion, collision checking, learner evaluation, and connection to a physical robot.

References

[1] J. J. Craig, Introduction to Robotics: Mechanics and Control, 3rd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005.

[2] P. Corke, W. Jachimczyk, and R. Pillat, Robotics, Vision and Control: Fundamental Algorithms in MATLAB, 3rd ed. Cham, Switzerland: Springer, 2023, doi: 10.1007/978-3-031-07262-8.

[3] The MathWorks, Inc., "rigidBodyTree: Create tree-structured robot," MATLAB Documentation. Accessed: Oct. 12, 2026. [Online]. Available: https://www.mathworks.com/help/robotics/ref/rigidbodytree.html

[4] The MathWorks, Inc., "getTransform: Get transform between body frames," MATLAB Documentation. Accessed: Oct. 12, 2026. [Online]. Available: https://www.mathworks.com/help/robotics/ref/rigidbodytree.gettransform.html